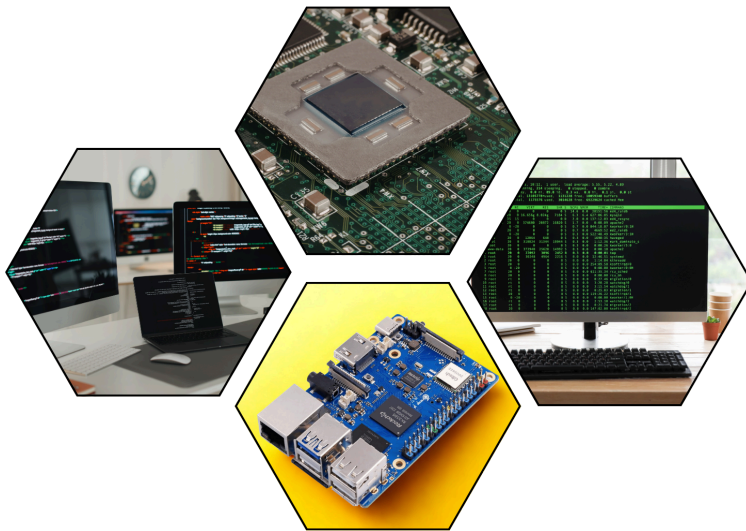


PROPOSAL PENELITIAN

Framework Penjadwalan Dinamis Adaptif berbasis Runtime Cost Estimation untuk Pipeline Multithreading pada Arsitektur Asimetris (Studi Kasus: Fast Super-Resolution Convolutional Neural Network)

A Runtime Cost Estimation based Adaptive Dynamic Scheduling Framework for Pipeline Multithreading on Asymmetric Architectures: A Case Study of Fast Super-Resolution Convolutional Neural Networks.



M. HAMDANI ILHAM LATJORO

D082252019



**PROGRAM STUDI MAGISTER TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS HASANUDDIN
GOWA
2026**

LEMBAR PENGESAHAN PROPOSAL PENELITIAN

***FRAMEWORK PENJADWALAN DINAMIS ADAPTIF BERBASIS RUNTIME
COST ESTIMATION UNTUK PIPELINE MULTITHREADING PADA ARSITEKTUR
ASIMETRIS (STUDI KASUS: FAST SUPER-RESOLUTION CONVOLUTIONAL
NEURAL NETWORK)***

**M. Hamdani Ilham Latjoro
D082252019**

Disetujui untuk diseminarkan

Komisi Pembimbing

Pembimbing I



Prof. Dr. Adnan, S.P., M.T.
NIP: 19740426 200501 1 002

Ketua Program Studi S2 Informatika

Prof. Dr. Ir. Zahir Zainuddin, M.Sc
NIP: 19640427 198910 1 002

DAFTAR ISI

	Halaman
LEMBAR PENGESAHAN PROPOSAL PENELITIAN.....	i
DAFTAR ISI.....	ii
DAFTAR TABEL.....	iv
DAFTAR GAMBAR.....	v
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tinjauan Pustaka.....	4
1.2.1 Hukum Amdahl dan Hukum Gustafson.....	4
1.2.1.1 Hukum Amdahl: Paradigma Beban Tetap.....	4
1.2.1.2 Hukum Gustafson: Paradigma Beban Skalabel.....	4
1.2.1.3 Implikasi untuk Penjadwalan pada Arsitektur Asimetris.....	5
1.2.2 Arsitektur Asimetris dan ARM <i>big.LITTLE</i>	5
1.2.2.1 Konsep Arsitektur Asimetris.....	5
1.2.2.2 Arsitektur ARM <i>big.LITTLE</i>	5
1.2.3 Konsep <i>Pipeline</i> dan Ketidakseimbangan Beban.....	7
1.2.3.1 Definisi dan Karakteristik <i>Pipeline</i>	7
1.2.3.2 Tantangan Ketidakseimbangan Beban dalam <i>Pipeline</i>	8
1.2.3.3 Pendekatan Penjadwalan: Statis dan Dinamis.....	8
1.2.4 Estimasi Biaya Komputasi Dinamis (<i>Runtime Cost Estimation</i>).....	9
1.2.4.1 Konsep Dasar Estimasi Biaya Melalui Kalibrasi Awal.....	9
1.2.4.2 Penerapan Estimasi Biaya pada Arsitektur Asimetris.....	9
1.2.5 Fast Super-Resolution Convolutional Neural Network (FSRCNN)....	10
1.2.5.1 Super-Resolusi Citra.....	10
1.2.5.2 Evolusi dari SRCNN ke FSRCNN.....	10
1.2.5.3 Arsitektur FSRCNN.....	11
1.2.5.4 Karakteristik Beban Komputasi FSRCNN.....	12
1.3 Rumusan Masalah.....	12
1.4 Hipotesa Penelitian.....	13
1.5 Tujuan Penelitian.....	13
1.6 Manfaat Penelitian.....	13
1.7 Batasan Masalah.....	14
BAB II METODOLOGI PENELITIAN.....	15
2.1 Jenis Penelitian.....	15
2.2 Waktu & Lokasi Penelitian.....	15
2.3 Tahapan Penelitian.....	15
2.4 Perangkat Penelitian.....	17
2.5 Perancangan Sistem.....	19
2.5.1 Gambaran Umum Arsitektur.....	19

2.5.2 Perancangan Komponen Utama.....	19
2.5.3 Integrasi Studi Kasus (FSRCNN).....	21
2.6 Definisi Operasional Variabel.....	21
2.7 Skenario Percobaan.....	22
2.8 Analisis Data.....	22
DAFTAR PUSTAKA.....	24
LAMPIRAN.....	27
Lampiran 1. <i>State of the Arts</i>	28

DAFTAR TABEL

	Halaman
Tabel 1. Lapisan FSRCNN.....	11
Tabel 2. Daftar Perangkat Keras.....	17
Tabel 3. Daftar Perangkat Lunak.....	17
Tabel 4. Definisi Operasional Variabel.....	20
Tabel 5. Skenario Percobaan.....	21

DAFTAR GAMBAR

	Halaman
Gambar 1. Laundry analogi untuk <i>pipeline</i>	7
Gambar 2. Arsitektur FSRCNN.....	11
Gambar 3. Tahapan Penelitain.....	16
Gambar 4. Gambaran Umum Arsitektur.....	19

BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan pesat di bidang kecerdasan buatan dan pemrosesan multimedia telah mendorong kebutuhan akan sistem komputasi yang mampu menangani beban kerja kompleks secara efisien. Yoo (2015) menjelaskan bahwa prosesor asimetris seperti ARM *big.LITTLE* mengombinasikan sejumlah inti berkinerja tinggi dengan inti berdaya rendah dalam satu chip untuk mencapai efisiensi energi sekaligus mempertahankan kinerja tinggi sehingga menjadi pilihan utama pada perangkat bergerak hingga *single-board computer* seperti Orange Pi 5. Namun, pemanfaatan sumber daya secara optimal pada arsitektur ini masih menghadapi tantangan fundamental, terutama ketidakseimbangan beban kerja (*load imbalance*) yang menyebabkan beberapa inti menganggur sementara yang lain mengalami kelebihan beban.

Dalam konteks penjadwalan pada AMP, paradigma yang selama ini mendominasi adalah Hukum Amdahl. Amdahl (1967) menyatakan bahwa percepatan (*speedup*) suatu program dibatasi oleh bagian sekuensial yang tidak dapat diparalelkan. Implikasinya, strategi penjadwalan berfokus pada minimalisasi latensi jalur kritis (*span*) dengan mengalokasikan tugas berat ke *big core*, sementara *LITTLE core* hanya digunakan untuk tugas latar belakang yang ringan. Pendekatan ini terbukti efektif untuk beban kerja dengan karakteristik tetap (*fixed workload*), seperti mempercepat eksekusi satu program tertentu.

Sebaliknya, Hukum Gustafson menawarkan perspektif yang lebih relevan untuk beban kerja skalabel. Gustafson (1988) berargumen bahwa ketika tersedia lebih banyak sumber daya komputasi, pengguna cenderung menyelesaikan masalah yang lebih besar dalam waktu yang sama, bukan masalah yang sama dalam waktu lebih singkat. Dengan kata lain, *throughput* dapat ditingkatkan secara proporsional terhadap jumlah sumber daya yang tersedia, selama beban kerja dapat diskalakan. Peralihan paradigma dari *latency-oriented* (Amdahl, 1967) ke *throughput-oriented* (Gustafson, 1988) menjadi kunci dalam mengoptimalkan eksekusi aplikasi kontinu pada arsitektur asimetris.

Paradigma pemrograman *pipeline* telah lama dikenal sebagai pendekatan efektif untuk mengeksploitasi paralelisme tingkat tugas, data, dan *pipeline* secara simultan. Moreno et al. (2017) dalam pengembangan HeDPM (Heterogeneous Dynamic Pipeline Mapping) menjelaskan bahwa algoritma ini menangani beban kerja aplikasi pada sistem heterogen melalui teknik *replication* (menduplikasi tahap lambat di beberapa prosesor) dan *gathering* (menggabungkan tahap cepat pada prosesor yang sama), mempertimbangkan kapasitas komputasi prosesor dan biaya komunikasi. Pendekatan ini menunjukkan peningkatan kinerja signifikan (hingga 9,2 kali pada *pipeline* Ferret dalam *benchmark* PARSEC) dengan kompleksitas rendah

$O(N^3)$, menjadikannya sangat efisien untuk penalaan dinamis saat aplikasi berjalan (*runtime*).

Meskipun sangat efektif, HeDPM memiliki keterbatasan di mana model kinerjanya bergantung pada pengetahuan awal mengenai struktur aplikasi seperti jumlah dan urutan tahap *pipeline* agar dapat mendefinisikan fungsi kinerja sebelum diintegrasikan ke dalam alat penalaan. Selain itu, karena HeDPM secara eksplisit memodelkan biaya komunikasi antarprosesor yang terpisah, algoritma ini lebih dioptimalkan untuk sistem terdistribusi (seperti *grid* atau *cloud*) dan kurang relevan untuk arsitektur asimetris dalam satu chip seperti pada *big.LITTLE* di mana memori digunakan bersama dan *overhead* komunikasi sangat rendah.

Penelitian terdahulu oleh Chronaki et al. (2017) telah menunjukkan bahwa pada arsitektur *multi-core* asimetris, pendekatan penjadwalan dinamis yang mengevaluasi jalur kritis (*critical path*) mampu meningkatkan performa sistem hingga 2,1 kali dibandingkan penjadwal standar. Dalam pengujiannya, mereka menggunakan 5 aplikasi salah satunya Bodytrack yang secara spesifik telah menerapkan model *pipeline* paralel dua tahap yang disinkronisasi melalui anotasi dataflow OmpSs. Pada implementasi tersebut, penjadwal (CPATH dan HYBRID) bekerja dengan mendeteksi tugas-tugas kritis di dalam *pipeline* dan memprioritaskannya ke *big core* untuk mengurangi total waktu eksekusi aplikasi secara keseluruhan.

Namun, meskipun telah menerapkan *pipeline*, pendekatan tersebut masih memiliki celah signifikan karena optimasinya bersifat *throughput-agnostic*, di mana fokus utamanya adalah mempercepat penyelesaian seluruh tugas (408.525 tugas pada Bodytrack) tanpa adanya mekanisme untuk menjamin batas *throughput real-time* yang konstan. Selain itu, karena Bodytrack memiliki variabilitas biaya tugas yang sangat rendah (99% tugas memiliki waktu eksekusi serupa), penjadwal tersebut tidak dirancang untuk menangani dinamika seperti backpressure dan thread starvation yang terjadi antar-tahap *pipeline* yang lebih kompleks pada pemrosesan video. Akibatnya, sistem cenderung memaksimalkan performa *big core* secara agresif demi *speedup*, yang memicu pemborosan energi karena tidak adanya kesadaran untuk menurunkan beban kerja ke *little core* ketika target sebenarnya sudah terpenuhi.

Sebagai upaya mengatasi tantangan pada arsitektur asimetris, Wang et al. (2020) memperkenalkan *framework* Pipe-it yang secara spesifik dirancang untuk inferensi CNN pada prosesor ARM *big.LITTLE*. *Framework* ini mengadopsi strategi *pipeline partitioning* di mana lapisan-lapisan konvolusi dibagi dan dialokasikan ke kluster *core* yang berbeda (*Big* atau *LITTLE*) untuk memaksimalkan throughput. Keberhasilan Pipe-it dalam meningkatkan throughput hingga 39% bergantung pada sebuah model prediksi kinerja (*performance-prediction model*). Model ini memanfaatkan deskriptor dari lapisan konvolusi (seperti ukuran filter dan dimensi data) untuk memperkirakan waktu eksekusi. Prediksi ini kemudian digunakan untuk

merancang konfigurasi *pipeline* yang optimal melalui algoritma *design space exploration* sebelum eksekusi dimulai.

Meskipun efektif, pendekatan yang digunakan oleh Pipe-it bersifat statis (*design-time*). Keputusan alokasi tugas ditentukan di awal berdasarkan karakteristik model CNN yang diketahui. Ketergantungan pada *deskriptor* lapisan ini menyebabkan *framework* kurang fleksibel jika dihadapkan pada beban kerja dinamis yang karakteristiknya berubah saat *runtime* (misalnya perubahan resolusi video secara tiba-tiba) atau jika struktur aplikasi tidak sepenuhnya diketahui sebelumnya. Keterbatasan ini menjadi celah penelitian yang mendorong dikembangkannya pendekatan penjadwalan dinamis berbasis *Runtime Cost Estimation (RCE)* dalam penelitian ini.

Pada ranah yang berbeda, penelitian tentang DynPipe yang dikenalkan oleh Yuan (2025) mengusulkan *framework pipeline* asinkron yang mampu mempartisi ulang beban model secara dinamis (*real-time load balancing*) untuk pelatihan (*training*) model *deep learning* skala besar di lingkungan kluster multi-GPU. Pendekatan ini secara efektif mengatasi ketidakseimbangan beban akibat *straggler* melalui model *random forest* yang memprediksi dampak gangguan secara nonintrusif berdasarkan statistik *runtime*.

Meskipun adaptif secara dinamis, model kinerjanya masih bergantung pada pengetahuan awal mengenai struktur aplikasi (seperti urutan dan profil lapisan model) untuk menginisialisasi formulasi *pipeline* sebelum eksekusi dimulai. Selain itu, karena fokusnya adalah pada optimasi pelatihan di lingkungan kluster dengan biaya komunikasi eksplisit, strategi ini kurang sesuai untuk fase *inference* pada arsitektur asimetris dalam satu chip (seperti *big.LITTLE*) di mana memori digunakan bersama dan *overhead* komunikasi antartahap sangat rendah.

Sementara itu, penelitian paling mutakhir oleh Saad et al. (2026) mengusulkan *framework* QLSA-MOEAD yang memadukan Q-learning untuk inisialisasi adaptif, *Simulated Annealing* sebagai penyempurna pencarian lokal, serta MOEA/D dalam optimasi multi-objektif guna mengelola penjadwalan tugas pada lingkungan komputasi heterogen yang mencakup CPU, GPU, dan FPGA. Walaupun mampu merespons kedatangan tugas dinamis secara *real-time* pada kisaran submilidetik (0,80–1,70 ms), pendekatan ini masih terbatas dari sisi ragam heterogenitas karena lebih ditujukan pada infrastruktur berskala makro seperti *cloud* atau kluster, alih-alih arsitektur asimetris satu chip berbasis *shared memory* yang memiliki *overhead* komunikasi jauh lebih kecil. Di samping itu, QLSA-MOEAD dirancang untuk pola tugas berbentuk *Directed Acyclic Graph (DAG)*, sehingga belum sepenuhnya mengakomodasi *pipeline multithreading* dengan beban kerja yang dapat diskalakan, seperti pada FSRCNN, yang berupaya melampaui keterbatasan statis Hukum Amdahl.

Berdasarkan celah tersebut, penelitian ini mengusulkan pengembangan *framework* penjadwalan dinamis adaptif berbasis *Runtime Cost Estimation (RCE)* untuk *pipeline multithreading* pada arsitektur asimetris. Berbeda dengan metode

konvensional yang bergantung pada profiling manual atau pengetahuan awal (*prior knowledge*), *framework* ini dilengkapi dengan mekanisme umpan balik (*feedback loop*) yang mampu mengestimasi beban komputasi setiap tahap secara *real-time* saat aplikasi berjalan. Estimasi biaya ini kemudian digunakan untuk memetakan tugas secara cerdas, seperti tugas dengan beban komputasi tinggi dialokasikan secara preferensial ke *Big Core*, sementara tugas ringan dialokasikan ke *Little Core*. Dengan pendekatan ini, sistem mampu beradaptasi secara otonom terhadap variasi data (misalnya perubahan resolusi video) tanpa intervensi pengguna, sehingga memaksimalkan utilisasi sumber daya asimetris dan meningkatkan *throughput* secara optimal.

FSRCNN dipilih sebagai studi kasus karena karakteristik ketidakseimbangan beban alami antarlapisan sangat ideal untuk menguji kemampuan *framework* dalam merespons penumpukan data (*bottleneck*) secara dinamis guna mengoptimalkan *throughput* pada pemrosesan video. Dengan *framework* ini, diharapkan pemanfaatan sumber daya CPU pada arsitektur asimetris dapat dioptimalkan, mencegah fenomena *thread starvation*, dan *throughput* pemrosesan tidak lagi terbatas oleh asumsi beban tetap Hukum Amdahl. Hasil penelitian ini diharapkan dapat memberikan kontribusi berupa pola arsitektural umum untuk *pipeline* paralel adaptif yang dapat diterapkan pada berbagai domain aplikasi, mulai dari encoding video, *rendering* grafis, hingga pemrosesan sinyal *real-time*.

1.2 Tinjauan Pustaka

1.2.1 Hukum Amdahl dan Hukum Gustafson

1.2.1.1 Hukum Amdahl: Paradigma Beban Tetap

Hukum Amdahl dikemukakan oleh Gene Amdahl pada tahun 1967 dalam konferensi AFIPS. Hukum ini menjadi fondasi teoritis utama dalam memahami batasan *speedup* yang dapat dicapai melalui paralelisasi. Amdahl (1967) berargumen bahwa tidak semua bagian dari suatu program dapat diparalelkan, selalu ada bagian yang harus dijalankan secara sekuensial. Bagian sekuensial inilah yang menjadi faktor pembatas utama terhadap percepatan yang dapat dicapai, tidak peduli berapa banyak prosesor yang ditambahkan.

Secara matematis, Hukum Amdahl dirumuskan sebagai:

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Di mana:

S = *speedup* yang dicapai.

P = proporsi kode yang dapat diparalelkan ($0 \leq P \leq 1$)

$(1 - P)$ = proporsi kode yang harus dijalankan secara sekuensial

N = jumlah prosesor atau unit pemrosesan

Dari rumus ini, dapat dilihat bahwa ketika N mendekati tak terhingga, percepatan maksimum yang dapat dicapai adalah $S_{max} = 1 / (1 - P)$. Sebagai

contoh, jika 90% program dapat diparalelkan ($P = 0,9$), maka percepatan maksimum hanya 10 kali lipat, tidak peduli berapa banyak inti yang ditambahkan.

Asumsi fundamental yang mendasari Hukum Amdahl adalah beban kerja tetap (*fixed workload*). Amdahl mengasumsikan bahwa ukuran masalah atau volume data yang diproses tidak berubah ketika jumlah prosesor ditingkatkan. Tujuannya adalah mempercepat penyelesaian tugas yang sama dalam waktu yang lebih singkat.

1.2.1.2 Hukum Gustafson: Paradigma Beban Skalabel

Hukum Gustafson diperkenalkan oleh John L. Gustafson pada tahun 1988 sebagai kritik terhadap asumsi beban tetap yang digunakan dalam Hukum Amdahl. Gustafson (1988) berargumen bahwa dalam praktik komputasi nyata, ketika tersedia lebih banyak sumber daya komputasi, pengguna cenderung menyelesaikan masalah yang lebih besar dalam waktu yang sama, bukan menyelesaikan masalah yang sama dalam waktu yang lebih singkat.

Gustafson (1988) mengamati bahwa dalam berbagai aplikasi ilmiah dan rekayasa, peningkatan jumlah prosesor justru digunakan untuk meningkatkan resolusi, akurasi, atau volume data yang diproses, sambil mempertahankan waktu eksekusi yang konstan.

Berdasarkan observasi ini, Gustafson merumuskan hukum yang kemudian dikenal sebagai Hukum Gustafson atau *Gustafson's Law*:

$$S(N) = N + (1-N) s$$

Di mana:

$S(N)$ = *speedup* yang dicapai

N = jumlah prosesor

s = proporsi waktu eksekusi bagian serial

Persamaan ini menekankan bahwa pada komputasi paralel, ukuran masalah dapat diperbesar seiring bertambahnya jumlah prosesor, sehingga bagian paralel mendominasi waktu eksekusi dan keterbatasan yang dijelaskan oleh Amdahl menjadi tidak lagi dominan.

1.2.1.3 Implikasi untuk Penjadwalan pada Arsitektur Asimetris

Perbedaan antara kedua hukum ini memiliki implikasi signifikan terhadap strategi penjadwalan pada arsitektur asimetris. Penelitian oleh Hill dan Marty (2008) menunjukkan bahwa dalam era *multi-core*, Hukum Amdahl sering kali terlalu pesimistis karena mengabaikan fakta bahwa banyak aplikasi modern memiliki beban kerja yang dapat diskalakan.

Dalam konteks penelitian ini, FSRCNN merupakan aplikasi pemrosesan video kontinu di mana tujuan utamanya adalah memproses sebanyak mungkin frame per detik (*throughput*), bukan mempercepat pemrosesan satu frame. Oleh karena itu, paradigma Hukum Gustafson lebih relevan sebagai landasan penjadwalan dibandingkan dengan Hukum Amdahl.

1.2.2 Arsitektur Asimetris dan ARM *big.LITTLE*

1.2.2.1 Konsep Arsitektur Asimetris

Yoo (2015) mengemukakan bahwa arsitektur asimetris (*Asymmetric Multicore Processor/AMP*) adalah desain prosesor yang mengintegrasikan inti dengan karakteristik kinerja dan efisiensi daya yang berbeda dalam satu chip. Berbeda dengan arsitektur homogen di mana semua inti identik, AMP memungkinkan perangkat untuk mengoptimalkan keseimbangan antara kinerja dan konsumsi energi berdasarkan kebutuhan komputasi yang dinamis.

Motivasi utama pengembangan arsitektur asimetris berasal dari batasan fisik dan termal pada perangkat bergerak. Dengan meningkatnya jumlah inti, konsumsi daya dan panas yang dihasilkan menjadi tantangan utama. Arsitektur asimetris menawarkan solusi dengan menyediakan inti hemat daya untuk tugas-tugas ringan dan inti berkinerja tinggi untuk beban komputasi berat.

1.2.2.2 Arsitektur ARM *big.LITTLE*

Al-Shaikh et al. (2023) menjelaskan bahwa desain mikroprosesor *modern* sangat terbantu oleh penerapan arsitektur komputasi asimetris seperti ARM *big.LITTLE*. Arsitektur ini menggabungkan beberapa inti besar (*big cores*) yang berfokus pada kinerja tinggi dengan inti kecil (*little cores*) yang dioptimalkan untuk hemat energi. Tujuan utama dari penggabungan ini adalah untuk menciptakan prosesor *multi-core* yang mampu mengatasi dua tantangan utama dalam desain sistem mobile maupun tertanam (*embedded*), yakni mencapai performa komputasi yang maksimal sekaligus menjaga konsumsi daya tetap rendah.

Salah satu implementasi arsitektur asimetris yang paling luas diadopsi adalah ARM (2026) *big.LITTLE*, yang menjadi dasar dari prosesor yang digunakan dalam penelitian ini yaitu Rockchip RK3588S pada Orange Pi 5. Teknologi *big.LITTLE* mengkombinasikan dua jenis inti prosesor dalam satu chip:

- a. *Core* besar (*big core*): dirancang untuk kinerja tinggi, biasanya dengan arsitektur mikro yang kompleks, frekuensi lebih tinggi, dan konsumsi daya lebih besar. Contoh: Cortex-A76.
- b. *Core* kecil (*LITTLE core*): Dioptimalkan untuk efisiensi daya, dengan arsitektur lebih sederhana, frekuensi lebih rendah, tetapi konsumsi daya jauh lebih rendah. Contoh: Cortex-A55.

Tujuan utama dari arsitektur *big.LITTLE* mencapai keseimbangan optimal antara kinerja dan efisiensi energi. Tugas-tugas komputasi berat dialokasikan ke *core* besar untuk penyelesaian cepat, sementara tugas latar belakang atau beban ringan ditangani oleh *core* kecil untuk menghemat daya. Namun, penjadwalan *thread* yang efektif pada arsitektur asimetris ini menghadapi tantangan tersendiri karena sulitnya memprediksi performa setiap *thread* pada setiap tipe inti serta mengidentifikasi pola komunikasi dan *thread* yang menjadi *bottleneck*.

Dalam konteks penelitian ini, meskipun perangkat target (Orange Pi 5) memiliki arsitektur *big.LITTLE*, *framework* bersifat *asymmetry-aware*. Misalnya, *worker pool*

harus bisa mengalokasikan antrean tugas berat (seperti dekonvolusi) secara eksklusif atau memprioritaskannya ke *big core*, sementara tugas ringan ke *LITTLE core*. Keputusan ini diambil untuk menyederhanakan model penjadwalan dan membuktikan efektivitas *Runtime Cost Estimation (RCE)* bahkan tanpa eksploitasi penuh terhadap heterogenitas arsitektur.

1.2.2.3 Tantangan Pemrograman pada Sistem Asimetris

Pemrograman paralel pada sistem asimetris membawa tantangan klasik yang diidentifikasi oleh Yu et al. (2020), antara lain:

- a. Kondisi balapan (*race condition*) terjadi ketika beberapa *thread* mengakses data bersama tanpa sinkronisasi yang memadai.
- b. Ketidakseimbangan beban. Dalam paradigma *pipeline computing*, ketidakseimbangan durasi eksekusi antar-tahap memicu masalah sistemik berupa *backpressure* (tekanan balik akibat antrean tahap yang lambat penuh) dan *starvation* (kelaparan data pada tahap yang lebih cepat).
- c. Penjadwalan adaptif memerlukan mekanisme yang mampu mencocokkan karakteristik thread dengan jenis inti prosesor yang tersedia.

Penelitian ini secara khusus berfokus pada penyelesaian masalah ketidakseimbangan beban melalui pendekatan *Runtime Cost Estimation (RCE)* yang diimplementasikan dalam *framework pipeline multithreading*.

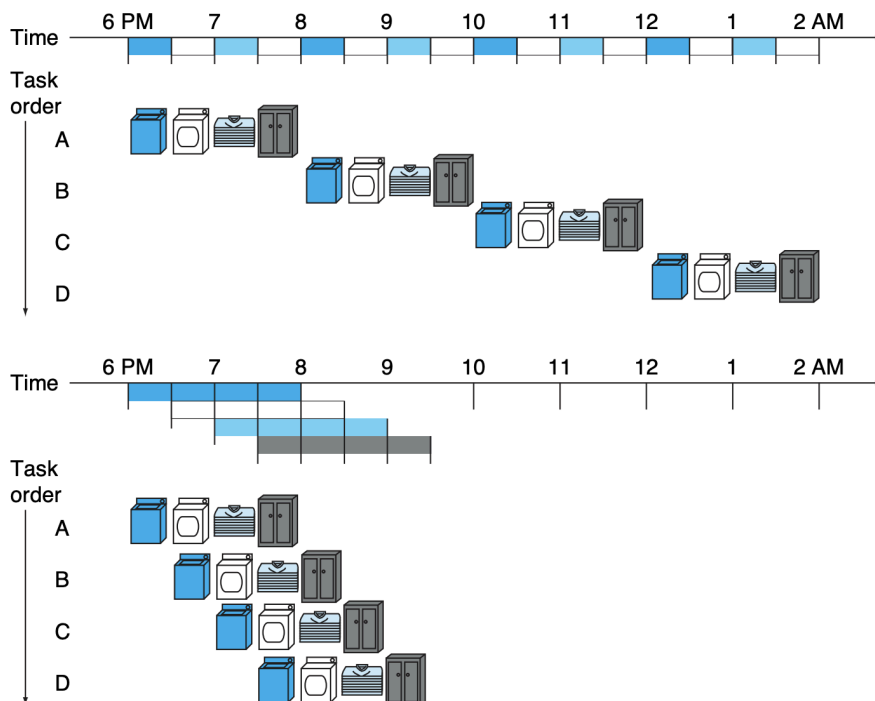
1.2.3 Konsep Pipeline dan Ketidakseimbangan Beban

1.2.3.1 Definisi dan Karakteristik Pipeline

Secara fundamental, *pipelining* adalah teknik arsitektural di mana beberapa instruksi atau tugas dieksekusi secara tumpang tindih (*overlapping*). Menurut Patterson dan Hennessy (2014), proses komputasi dalam *pipeline* dipecah menjadi serangkaian langkah independen yang disebut sebagai tahap (*stage*). Selama terdapat sumber daya pemrosesan yang terpisah untuk masing-masing tahap, seluruh tahap dapat beroperasi secara konkuren. Karakteristik unik dari arsitektur ini memunculkan sebuah paradoks *pipelining* yaitu waktu eksekusi untuk menyelesaikan satu tugas tunggal dari awal hingga akhir (*latency*) tidak menjadi lebih singkat. Akan tetapi, karena seluruh tahap bekerja secara paralel terhadap aliran data yang berbeda, sistem mampu menyelesaikan lebih banyak beban kerja dalam satu satuan waktu. Dengan demikian, *pipelining* secara signifikan meningkatkan laju penyelesaian kerja (*throughput*) sistem secara keseluruhan, yang berujung pada penurunan drastis total waktu pemrosesan ketika menangani volume data yang besar, seperti pada pemrosesan bingkai video (*video frame*).

Untuk memperjelas mekanisme kerja *pipeline*, Patterson dan Hennessy (2014) menggunakan analogi pembersihan pakaian (*laundry*) seperti yang diilustrasikan pada Gambar 1. Dalam analogi ini, setiap tahap pemrosesan (mencuci, mengeringkan, melipat, dan menyimpan) mewakili sebuah *stage* dalam *pipeline*.

Poin krusial dari analogi ini adalah bahwa *pipelining* tidak mempercepat waktu yang dibutuhkan untuk menyelesaikan satu beban cucian tunggal (*latency*), melainkan meningkatkan jumlah cucian yang dapat diselesaikan dalam jangka waktu tertentu (*throughput*). Dalam konteks penelitian ini, setiap tahap pada Gambar 1 tersebut berkorespondensi dengan lapisan-lapisan pada model FSRCNN, di mana optimalisasi *throughput* menjadi tujuan utama agar sistem dapat memproses video secara *real-time*.



Gambar 1. Laundry analogi untuk *pipeline* (Patterson dan Hennessy, 2014)

Mastoras, A., & Gross, T. (2018) menyebutkan bahwa *pipeline* adalah teknik paralelisasi yang memungkinkan eksekusi simultan dari *loop-loop* yang memiliki ketergantungan antariterasi (*cross-iteration dependences*). Dalam model *pipeline*, iterasi *loop* dipartisi menjadi serangkaian tahap berurutan, dan paralelisme muncul dari eksekusi tumpang tindih dari iterasi *loop* yang berbeda. Data mengalir dari tahap awal ke tahap akhir, membentuk apa yang disebut sebagai *pipeline* linier (*linear pipeline*).

1.2.3.2 Tantangan Ketidakseimbangan Beban dalam *Pipeline*

Meskipun *pipeline* menjanjikan peningkatan kinerja, implementasinya menghadapi tantangan serius terkait ketidakseimbangan beban. Mastoras (2019) dalam disertasinya menjelaskan bahwa teknik *pipelining* yang mengandalkan

algoritma penjadwalan statis berkinerja buruk untuk *pipeline* linier dengan beban tidak seimbang. Ketidakseimbangan ini dapat disebabkan oleh:

- a. Variasi waktu eksekusi antartahap: beberapa tahap secara inheren lebih berat secara komputasi dibandingkan dengan tahap lain.
- b. Variasi data: karakteristik data masukan yang berbeda menyebabkan waktu pemrosesan bervariasi untuk tahap yang sama.
- c. Ketidakpastian sistem: faktor seperti *cache miss*, *page fault*, dan interferensi sistem operasi.

Dalam *pipeline* statis dengan satu *thread* per tahap, tahap-tahap cepat akan banyak menganggur menunggu tahap lambat (*bottleneck*), sehingga utilisasi CPU menjadi tidak optimal.

1.2.3.3 Pendekatan Penjadwalan: Statis dan Dinamis

Dalam sistem operasi, penjadwalan bertanggung jawab untuk menentukan proses mana yang berhak menggunakan CPU pada suatu waktu (Tanenbaum, 2015). Secara umum, terdapat dua pendekatan utama dalam penjadwalan. Penjadwalan statis mengalokasikan sumber daya dan menentukan urutan eksekusi sebelum program dijalankan. Pendekatan ini cenderung lebih efisien karena meminimalkan overhead akibat perpindahan proses (*context switch*) yang mahal biayanya (Tanenbaum, 2015). Namun, pendekatan ini tidak adaptif terhadap variasi *runtime*.

Penjadwalan dinamis, sebaliknya, membuat keputusan alokasi sumber daya saat program berjalan berdasarkan kondisi aktual. Penelitian sebelumnya yang dilakukan oleh Min et al. (2013) tentang penjadwalan dinamis untuk program *stream* irregular menunjukkan bahwa penjadwalan statis konvensional menghadapi keterbatasan skalabilitas pada sistem *many-core*. Penjadwal *dynamic load-balancing* yang mengeksplorasi hubungan *producer-consumer* dalam program *stream* mampu mencapai skalabilitas hampir linier dan mengungguli *runtime* paralel mutakhir hingga 2,8 kali lipat.

1.2.4 Estimasi Biaya Komputasi Dinamis (*Runtime Cost Estimation*)

1.2.4.1 Konsep Dasar Estimasi Biaya Melalui Kalibrasi Awal

Dalam konteks penjadwalan sistem operasi, pengetahuan mengenai durasi eksekusi suatu tugas merupakan faktor krusial untuk mencapai optimalisasi kinerja, khususnya pada algoritma penjadwalan seperti *Shortest-Job-First (SJF)*. Menurut Silberschatz et al. (2018), tantangan utama adalah bagaimana memperoleh estimasi durasi tersebut tanpa menimbulkan overhead yang besar pada sistem.

Pada banyak sistem komputasi modern, khususnya pemrosesan data sekuensial seperti *pipeline CNN*, beban kerja cenderung bersifat deterministik ketika parameter masukan (seperti resolusi gambar) tetap. Hal ini berarti waktu eksekusi setiap tahap relatif konstan dari satu iterasi ke iterasi berikutnya.

Berdasarkan prinsip deterministik ini, metode estimasi biaya tidak perlu dilakukan secara terus-menerus (*continuous monitoring*) pada setiap iterasi. Pengukuran yang dilakukan secara berulang kali menimbulkan *overhead* komputasi akibat pemanggilan fungsi *system call* (seperti *get_time*) yang berlebihan. Sebagai alternatif, diterapkan konsep kalibrasi awal (*initial calibration*), di mana pengukuran waktu eksekusi aktual hanya dilakukan sekali pada fase awal (*warm-up*) menggunakan satu sampel data representatif. Hasil pengukuran ini kemudian digunakan sebagai nilai estimasi biaya tetap (*cost*) untuk proses penjadwalan selanjutnya. Pendekatan ini secara efektif mengeliminasi *overhead runtime* yang biasanya terjadi pada metode prediksi dinamis konvensional.

1.2.4.2 Penerapan Estimasi Biaya pada Arsitektur Asimetris

Pada sistem komputasi asimetris seperti arsitektur ARM *big.LITTLE*, estimasi biaya komputasi menjadi instrumen vital dalam menentukan penempatan tugas pada jenis inti prosesor yang tepat. Wang et al. (2020) melalui *framework* Pipe-it membuktikan bahwa penggunaan model prediksi performa mampu meningkatkan *throughput* hingga 39% melalui partisi *pipeline* yang terencana secara statis. Namun, pendekatan *design-time* ini rentan mengalami kegagalan fungsional manakala kapasitas komputasi aktual dari perangkat keras tidak selaras dengan asumsi teoritis saat fase desain. Dalam struktur *pipeline*, ketidaksesuaian alokasi ini berujung pada gangguan sistemik, dimana tahap yang mendapatkan beban di luar kapasitas nyatanya akan memicu penumpukan antrean (*backpressure*) yang menyumbat tahap sebelumnya, sekaligus menyebabkan kelaparan data (*starvation*) pada tahap berikutnya yang pada akhirnya melumpuhkan laju pemrosesan secara keseluruhan.

Sebagai alternatif dari kekakuan model statis, Chronaki et al (2017). mengusulkan teknik penjadwalan dinamis yang mengevaluasi jalur kritis atau pemilihan eksekutor tercepat untuk mengoptimalkan waktu eksekusi pada sistem asimetris secara otomatis. Penelitian tersebut juga menyoroti efektivitas penjadwalan dinamis pada aplikasi dengan struktur *pipeline*, seperti pada *benchmark* Bodytrack yang memisahkan proses menjadi dua tahap utama, yaitu tahap penanganan input dan tahap komputasi. Melalui pemanfaatan informasi yang ditemukan selama masa eksekusi, pendekatan ini mampu menyesuaikan penugasan tugas berdasarkan ketersediaan inti prosesor tanpa memerlukan profil luar jaringan (*offline profiling*). Meskipun efektif dalam menangani ketergantungan tugas yang kompleks, mekanisme pelacakan jalur kritis secara kontinu pada setiap tahap eksekusi berpotensi menimbulkan *overhead* manajemen tugas yang cukup tinggi. Hal ini menjadi tantangan tersendiri pada sistem *streaming pipeline* yang membutuhkan latensi rendah, di mana kompleksitas penjadwalan dinamis yang terlalu berat justru dapat mengurangi efisiensi pada perangkat *embedded* dengan sumber daya terbatas.

Menjembatani kesenjangan antara kerentanan model statis dan kompleksitas penjadwalan dinamis berbasis jalur kritis, penelitian ini mengusulkan *Runtime Cost Estimation (RCE)* yang mengadopsi prinsip kalibrasi awal pada fase *warm-up*. Alih-alih melakukan pemantauan atau evaluasi jalur tugas yang berat secara terus-menerus selama program berjalan, RCE dirancang untuk melakukan estimasi biaya secara mandiri (*self-learning*) tepat sebelum fase pemrosesan data utama dimulai. Fase kalibrasi ini berfungsi untuk memastikan bahwa beban setiap tahap *pipeline* diukur secara nyata pada perangkat target, sehingga alokasi *thread* ke *big core* atau *LITTLE core* dapat dipetakan secara proporsional. Melalui pendekatan kalibrasi di awal eksekusi ini, risiko *backpressure* dan *starvation* dapat diminimalisir secara akurat tanpa membebani sistem dengan *overhead* pemantauan kontinu yang mengonsumsi sumber daya tinggi selama masa produksi data.

1.2.5 Fast Super-Resolution Convolutional Neural Network (FSRCNN)

1.2.5.1 Super-Resolusi Citra

Menurut Joseph dan Jayakumar (2024), super-resolusi (SR) adalah teknik peningkatan kualitas gambar yang didasari oleh proses pemulihan citra atau video beresolusi tinggi dari versi beresolusi rendah dan terkompresi. Kebutuhan akan citra super-resolusi berpengaruh pada berbagai aplikasi seperti *high-definition television*, perangkat pengawasan, pencitraan medis, satelit, dan pengenalan wajah.

Secara matematis, proses degradasi dari citra resolusi tinggi (HR) x ke citra resolusi rendah (LR) y dapat dimodelkan sebagai (Joseph & Jayakumar, 2024):

$$y = (x \otimes k) \downarrow s + n$$

Di mana $(x \otimes k)$ adalah operasi konvolusi antara citra HR x dan kernel blur k , $\downarrow s$ adalah operasi *downsampling* dengan faktor skala s , dan n merepresentasikan *additive white Gaussian noise (AWGN)*. Tujuan dari super-resolusi adalah membalikkan model degradasi ini untuk merekonstruksi citra HR dari observasi LR.

1.2.5.2 Evolusi dari SRCNN ke FSRCNN

Super-Resolution Convolutional Neural Network (SRCNN) yang diperkenalkan oleh Dong et al. (2015) merupakan pionir dalam penerapan deep learning untuk super-resolusi. SRCNN menunjukkan kinerja superior dibandingkan dengan metode konvensional, baik dalam kecepatan maupun kualitas restorasi. Namun, biaya komputasi yang tinggi masih menghalangi penggunaannya untuk aplikasi yang membutuhkan kinerja *real-time* (>24 FPS).

Untuk mengatasi keterbatasan ini, Dong et al. (2016) mengusulkan Fast Super-Resolution Convolutional Neural Network (FSRCNN) dengan tiga inovasi utama:

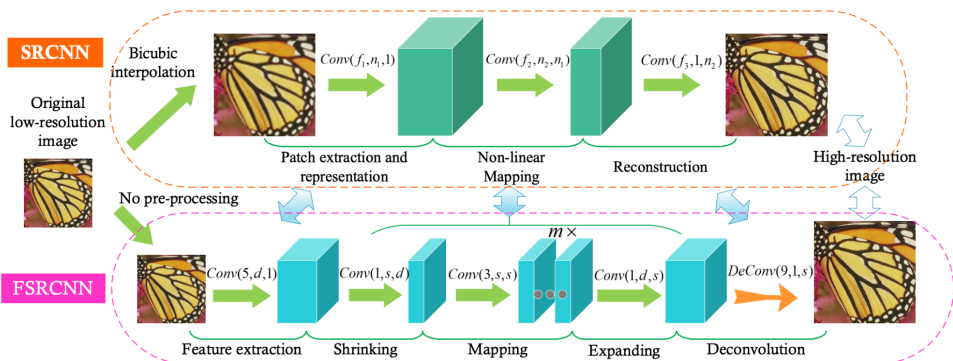
- a. Penggunaan lapisan dekonvolusi di akhir jaringan memungkinkan pembelajaran pemetaan langsung dari citra resolusi rendah asli (tanpa interpolasi) ke citra resolusi tinggi.

- b. Struktur hourglass menyusutkan dimensi fitur masukan sebelum pemetaan dan memperluasnya kembali setelahnya, sehingga mengurangi kompleksitas komputasi.
- c. Filter lebih kecil dengan lapisan pemetaan lebih banyak menggunakan ukuran filter yang lebih kecil, namun dengan jumlah lapisan yang lebih banyak untuk mempertahankan kualitas.

1.2.5.3 Arsitektur FSRCNN

Arsitektur FSRCNN terdiri dari beberapa bagian utama (Gambar 2):

- a. *Feature extraction*: lapisan konvolusi untuk mengekstraksi fitur dari citra beresolusi rendah.
- b. *Shrinking*: lapisan yang mengurangi dimensi fitur untuk efisiensi komputasi.
- c. *Mapping*: serangkaian lapisan konvolusi yang memetakan fitur ke representasi yang lebih kaya.
- d. *Expanding*: lapisan yang memperluas kembali dimensi fitur.
- e. *Deconvolution*: lapisan akhir yang melakukan *upscaling* untuk menghasilkan citra beresolusi tinggi.



Gambar 2. Arsitektur FSRCNN (Dong et al., 2016)

Penelitian terkini mengenai implementasi FSRCNN pada arsitektur asimetris telah dilakukan oleh Annisa et al. (2025). Penelitian tersebut menganalisis performa paralelisasi FSRCNN pada prosesor *big.LITTLE* dengan membandingkan beberapa skenario alokasi inti statis. Hasil pengujian menunjukkan bahwa skenario menggunakan 4 *big core* menghasilkan kualitas video dan waktu komputasi terbaik, sementara skenario hibrida (*big.LITTLE*) menawarkan keseimbangan, namun belum mampu menyamai performa *big core*.

Temuan krusial dalam penelitian tersebut mengidentifikasi adanya *bottleneck* komputasi yang signifikan pada *layer 8* (tahap dekonvolusi). Lapisan ini menunjukkan peningkatan *speedup* yang paling rendah dalam skenario paralel, menjadi faktor pembatas utama kecepatan keseluruhan *pipeline*. Keterbatasan penelitian tersebut yang masih menggunakan pendekatan skenario alokasi inti yang statis (tetap) menjadi landasan empiris yang kuat bagi pengembangan *framework*

penjadwalan dinamis dalam penelitian ini, yang dirancang untuk mengatasi ketidakseimbangan beban spesifik pada tahap dekonvolusi secara adaptif.

1.2.5.4 Karakteristik Beban Komputasi FSRCNN

FSRCNN memiliki karakteristik beban komputasi yang sangat bervariasi antar lapisan, menjadikannya studi kasus ideal untuk pengujian pada penelitian ini:

Tabel 1. Lapisan FSRCNN

Lapisan	Ukuran Filter	Jumlah Filter	Beban Komputasi
<i>Feature Extraction</i>	5x5	56	Sedang
<i>Shrinking</i>	1x1	12	Rendah
<i>Mapping (x4)</i>	3x3	12	Sedang
<i>Expanding</i>	1x1	56	Rendah
<i>Deconvolution</i>	9x9	1	Sangat Tinggi

Lapisan dekonvolusi di akhir jaringan memiliki beban komputasi jauh lebih berat dibandingkan dengan lapisan lainnya, menciptakan *bottleneck* alami dalam *pipeline* pemrosesan. Ketidakseimbangan inilah yang menjadi target utama *framework* berbasis *runtime cost estimation* yang dikembangkan dalam penelitian ini.

1.3 Rumusan Masalah

Jika terdapat sejumlah inti prosesor kinerja tinggi (*big core*) dan inti prosesor hemat daya (*little core*) yang tersedia untuk menjalankan pemrosesan *pipeline multithreading*, bagaimana mekanisme penjadwalan *CPU core* asimetris untuk meminimalkan konsumsi energi dengan tetap menjamin batas minimum *throughput real-time*, sebagaimana divalidasi pada studi kasus FSRCNN (24 FPS)?

1.4 Hipotesa Penelitian

Mekanisme penjadwalan *CPU core* asimetris yang menugaskan tahap *pipeline* berat ke *big core* dan tahap ringan ke *little core* secara adaptif berbasis *Runtime Cost Estimation (RCE)* meminimalkan konsumsi energi tanpa melanggar batas minimum *throughput real-time* (24 FPS) pada pemrosesan *pipeline* FSRCNN.

1.5 Tujuan Penelitian

Sesuai dengan rumusan masalah di atas, penelitian ini bertujuan untuk:

- a. Mengembangkan arsitektur *framework pipeline multithreading* yang mampu mendistribusikan beban kerja secara dinamis sesuai dengan kebutuhan setiap tahap *pipeline*.
- b. Mengembangkan sistem otomatis yang mengatur aliran data antartahap *pipeline* untuk mencegah penumpukan dan memastikan kelancaran pemrosesan serta mengoptimalkan energi.
- c. Merancang mekanisme penjadwalan adaptif berbasis *Runtime Cost Estimation (RCE)* sehingga secara cerdas dapat mengalokasikan beban kerja tanpa perlu melakukan *profiling* terhadap arsitektur asimetris sebelumnya.

1.6 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat sebagai berikut:

1.6.1 Manfaat Teoritis

- a. Memberikan kontribusi berupa pola arsitektural umum untuk *pipeline* paralel adaptif yang dapat menjadi rujukan dalam pengembangan sistem komputasi paralel pada berbagai domain aplikasi.
- b. Memperkaya khazanah ilmu pengetahuan di bidang *dynamic load balancing* khususnya dalam konteks *pipeline multithreading* dengan arsitektur asimetris.
- c. Menyediakan landasan teoretis bagi pengembangan lebih lanjut mekanisme prioritas dinamis berbasis *machine learning* pada *pipeline* paralel.

1.6.2 Manfaat Praktis

- a. Menghasilkan *framework* perangkat lunak yang siap digunakan oleh pengembang aplikasi *pipeline* untuk meningkatkan efisiensi pemrosesan data tanpa harus membangun mekanisme *offline profiling* dari awal.
- b. Memberikan solusi konkret bagi peningkatan kecepatan pemrosesan super-resolusi citra menggunakan FSRCNN, yang bermanfaat untuk aplikasi *real-time* seperti peningkatan kualitas video *streaming* atau citra medis.
- c. Menyediakan panduan implementasi *runtime cost estimation* yang dapat diadaptasi pada berbagai industri yang membutuhkan pemrosesan *pipeline* skala besar seperti *rendering* animasi, *encoding* video, dan pemrosesan sinyal radar.

1.7 Batasan Masalah

Agar penelitian ini lebih terfokus dan terarah, ditetapkan beberapa batasan sebagai berikut:

- a. **Arsitektur Target:** Implementasi dan pengujian *framework* difokuskan pada *single-board computer* Orange Pi 5 dengan SoC Rockchip RK3588S. SoC ini memiliki arsitektur ARM *big.LITTLE* yang asimetris terdiri dari 4 *core* Cortex-A76 (kinerja tinggi) dan 4 *core* Cortex-A55 (efisiensi daya).
- b. **Studi Kasus Utama:** Evaluasi kinerja dilakukan secara intensif pada aplikasi FSRCNN dengan faktor skala 2 kali untuk resolusi masukan 640x480 piksel (format QCIF) dan jumlah *frame* 150, sesuai dengan karakteristik video konferensi.
- c. **Paradigma Pipeline:** *Framework* dirancang untuk *pipeline* linier satu arah di mana setiap data melewati seluruh tahap secara berurutan, belum mencakup *pipeline* bercabang atau *cyclic pipeline*.
- d. **Bahasa Pemrograman:** Implementasi *framework* menggunakan bahasa C dengan pustaka POSIX Threads, sehingga penggunaannya terbatas pada sistem operasi berbasis Unix/Linux.
- e. **Parameter Prioritas:** Prioritas eksekusi tugas tidak ditetapkan secara statis (*hardcoded*), melainkan dievaluasi secara dinamis (*dynamic priority adjustment*) berdasarkan metrik panjang antrean (*queue length*) pada masing-masing tahap *pipeline* saat *runtime*.
- f. **Manajemen Memori:** Alokasi dan dealokasi memori untuk data pengguna menjadi tanggung jawab *framework*, sementara *framework* hanya mengelola memori untuk struktur internal dan pembungkus tugas.
- g. **Jumlah Tugas:** *Framework* mengasumsikan jumlah total tugas diketahui di awal eksekusi untuk keperluan alokasi *reorder buffer*, sehingga kurang optimal untuk skenario *streaming* dengan jumlah tugas tidak terbatas.

BAB II METODOLOGI PENELITIAN

2.1 Jenis Penelitian

Menurut OECD (2015), kegiatan Penelitian dan Pengembangan (R&D) mencakup tiga komponen utama, yaitu penelitian dasar, penelitian terapan, dan pengembangan eksperimental. Penelitian ini menggunakan pendekatan *applied research* yang bertujuan untuk menghasilkan produk berupa *framework* perangkat lunak sekaligus menguji efektivitasnya. Pendekatan ini dipilih karena sesuai dengan karakteristik penelitian yang tidak hanya mengembangkan artefak (*framework pipeline multithreading*), tetapi juga melakukan evaluasi kuantitatif terhadap kinerja artefak tersebut pada studi kasus nyata.

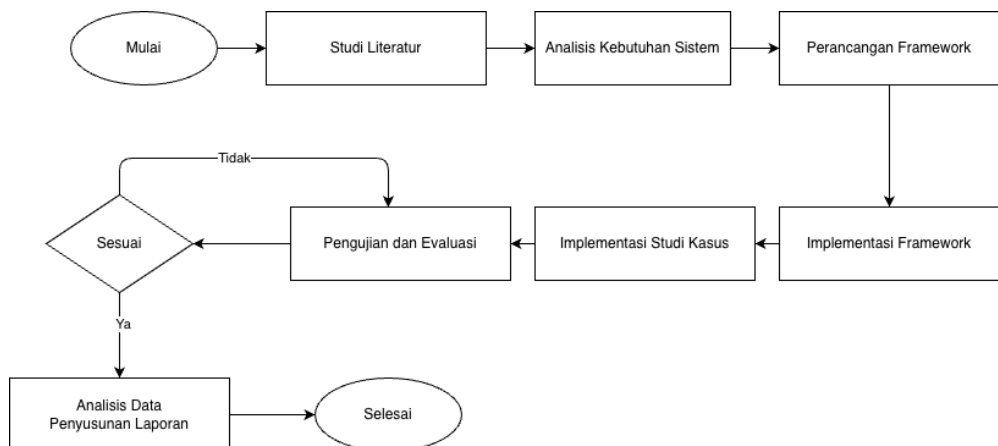
Jenis penelitian ini termasuk dalam kategori eksperimental komparatif, di mana performa *framework* yang dikembangkan dibandingkan dengan pendekatan *baseline (pipeline statis)* melalui pengukuran metrik kinerja terukur.

2.2 Waktu & Lokasi Penelitian

Penelitian ini dilaksanakan sejak bulan Februari hingga Desember 2026. Studi literatur dan perancangan *framework* dilakukan pada bulan Februari hingga awal April. Implementasi *framework* dan FSRCNN dilaksanakan pada bulan Mei hingga Juli, sedangkan pengujian, evaluasi, serta analisis hasil dilakukan pada bulan Agustus hingga Desember. Tahap akhir berupa revisi dan finalisasi laporan diselesaikan pada bulan Desember 2026.

Seluruh rangkaian penelitian bertempat di Laboratorium IOT & Komputasi Parallel, Fakultas Teknik, Universitas Hasanuddin. Proses pengembangan perangkat lunak dan pengujian kinerja menggunakan perangkat keras Orange Pi 5 dengan spesifikasi sebagaimana diuraikan pada subbab 2.4.1 serta perangkat lunak pendukung yang telah ditentukan.

2.3 Tahapan Penelitian



Gambar 3. Tahapan Penelitian

Tahapan penelitian disusun secara sistematis mengikuti model pengembangan R&D yang dikemukakan oleh OECD (2015), yaitu sebuah aktivitas R&D harus memenuhi lima kriteria utama: bersifat baru (*novel*), kreatif, memiliki ketidakpastian dalam hasil akhir (*uncertain*), dilakukan secara sistematis, serta dapat ditransfer atau direproduksi.

Adapun tahapan yang dilakukan dalam penelitian ini adalah sebagai berikut:

2.3.1 Studi Literatur

Tahap ini merupakan fondasi utama penelitian yang bertujuan untuk membangun kerangka pemikiran konseptual. Kegiatan mencakup pengumpulan dan kajian kritis terhadap sumber referensi primer (jurnal internasional bereputasi, *proceedings* konferensi, dan buku teks standar). Fokus kajian meliputi: arsitektur prosesor asimetris (ARM *big.LITTLE*), teori estimasi beban kerja sistem operasi (khususnya prinsip determinisme beban kerja dan konsep kalibrasi awal), teknik *pipeline parallelism* pada DNN, serta *state-of-the-art* penjadwalan dinamis (seperti Pipe-it dan DynPipe). Hasil dari tahap ini adalah identifikasi *research gap* terkait *overhead* komputasi pada mekanisme monitoring kontinu dan perumusan metode *initial calibration* yang diusulkan sebagai solusi ringan (*lightweight*) untuk perangkat *embedded*.

2.3.2 Analisis Kebutuhan Sistem

Pada tahap ini dilakukan identifikasi spesifikasi teknis perangkat keras dan perangkat lunak yang diperlukan. Di sisi hardware, dilakukan karakterisasi *platform* target (Orange Pi 5 dengan SoC RK3588S) meliputi jumlah inti fisik, topologi *cache*, dan kebijakan frekuensi CPU (DVFS). Di sisi software, ditentukan kebutuhan pustaka sistem (POSIX Threads), kompiler, serta format data input/output. Selain itu, dirumuskan metrik keberhasilan (*Key Performance Indicators*) seperti *throughput* (FPS), *latency per frame*, *CPU utilization*, dan *energy efficiency* (EDP).

2.3.3 Perancangan Framework

Tahap ini berfokus pada transformasi konsep teoritis menjadi cetak biru arsitektur perangkat lunak (*software blueprint*). Perancangan mencakup pendefinisian struktur data internal (seperti *Task Wrapper*, *Stage Queue*, dan *Reorder Buffer*), mekanisme sinkronisasi (*mutex* dan *condition variables*), serta arsitektur alur kerja (*workflow*) antarkomponen. Pada tahap ini, mekanisme *Runtime Cost Estimation* (RCE) diformulasikan melalui pendekatan kalibrasi awal (*initial calibration*) yang dilakukan sekali pada fase *warm-up* untuk menghasilkan nilai estimasi biaya tetap yang disimpan dalam *cost table* pada *shared memory*. Selain itu, dirancang pula logika *decision engine* bagi *worker thread* untuk memilih stage yang tepat berdasarkan estimasi biaya tersebut (*cost-aware decision*).

Dengan menetapkan biaya di awal, sistem dapat mengalokasikan sumber daya secara dinamis tanpa beban komputasi berlebih dari pemanggilan *system call* waktu yang berulang kali. Detail spesifikasi teknis arsitektur ini akan dibahas mendalam pada subbab terpisah (Perancangan Sistem).

2.3.4 Implementasi *Framework*

Proses penerjemahan rancangan ke dalam kode program yang dapat dieksekusi. Implementasi dilakukan menggunakan bahasa pemrograman C untuk memastikan efisiensi memori dan kedekatan dengan *system call* kernel Linux. Komponen utama yang dibangun meliputi *Worker Pool Manager* (mengatur afinitas *thread* ke *Big/Little core*), *Priority Scheduler* (algoritma pengambilan tugas dinamis), dan *Backpressure Controller* (mencegah *overflow* pada antrean). Selama implementasi, diterapkan praktik pemrograman paralel yang aman (*thread-safe*) untuk menghindari *race condition* dan *deadlock*.

2.3.5 Implementasi Studi Kasus FSRCNN

Tahap integrasi objek penelitian ke dalam *framework* yang telah dibangun. Model Fast Super-Resolution Convolutional Neural Network (FSRCNN) dipecah menjadi 8 tahap *pipeline* sesuai dengan arsitektur lapisannya. Fungsi pemrosesan setiap lapisan (*convolution*, *shrinking*, *mapping*, *deconvolution*) dikodekan sebagai fungsi *callback* yang akan dipanggil oleh *worker*. Selain itu, disiapkan modul pra-pemrosesan data (pembacaan file YUV, normalisasi piksel) dan pasca-pemrosesan (penulisan hasil super-resolusi).

2.3.6 Pengujian dan Evaluasi

Eksperimen dilaksanakan secara terkontrol di lingkungan laboratorium. Variabel bebas berupa skenario penjadwalan (Statis dan Dinamis) dan konfigurasi *worker* (jumlah *thread big.LITTLE*) divariasikan untuk mengukur variabel terikat (*throughput*, *latency*, utilisasi CPU). Data dikumpulkan secara kuantitatif menggunakan *profiling tools* standar Linux (seperti *perf* dan *time*). Jika hasil pengujian menunjukkan anomali atau inkonsistensi data, proses kembali ke tahap revisi dan optimasi untuk penyempurnaan algoritma.

2.3.7 Analisis dan Pelaporan

Data kuantitatif yang terkumpul dianalisis menggunakan metode statistik deskriptif dan komparatif. Tahap ini bertujuan membuktikan hipotesis penelitian bahwa penjadwalan *CPU core* asimetris yang menugaskan tahap *pipeline* berat ke *big core* dan tahap ringan ke *little core* secara adaptif berbasis *Runtime Cost Estimation (RCE)*. Analisis juga mencakup evaluasi efektivitas metode RCE dalam mengurangi *bottleneck* pada *layer* dekonvolusi dibandingkan dengan metode penjadwalan konvensional.

Tahap akhir mencakup kompilasi seluruh proses penelitian, hasil temuan, dan kesimpulan akademis ke dalam bentuk dokumen tesis. Penyusunan laporan mengikuti pedoman penulisan karya ilmiah standar, dilengkapi dengan visualisasi data (grafik dan tabel) serta rekomendasi untuk penelitian lanjutan.

2.4 Perangkat Penelitian

2.4.1 Perangkat Keras

Tabel 2. Daftar Perangkat Keras

Komponen	Spesifikasi	Fungsi
<i>Single Board Computer</i>	Orange Pi 5 (Rockchip RK3588S, 4x Cortex-A76 @2.4 GHz, 4x Cortex-A55 @1.8 GHz, RAM 8 GB)	<i>Platform</i> eksekusi utama
Media Penyimpanan	MicroSD 64 GB, SSD 128 GB	Penyimpanan sistem operasi, kode sumber, dan data uji
Perangkat Tambahan	Keyboard, mouse, monitor, kabel HDMI	Interaksi dan <i>debugging</i>

2.4.2 Perangkat Lunak

Tabel 3. Daftar Perangkat Lunak

Komponen	Spesifikasi	Fungsi
Sistem Operasi	Ubuntu 22.04 LTS (ARM64)	Lingkungan pengembangan dan eksekusi
Compiler	GCC 15.2.0 (dengan dukungan pthread)	Kompilasi kode sumber
<i>Profiling Tools</i>	htop, perf	Pengukuran utilisasi CPU dan waktu eksekusi
Instrumen Pengukuran Daya	PowerTop	Pemantauan konsumsi energi secara <i>real-time</i>
<i>Editor / IDE</i>	Visual Studio Code, Anti Gravity	Pengembangan kode

Version Control	Git & Github	Manajemen sumber	versi	kode
-----------------	--------------	------------------	-------	------

2.4.3 Data Uji

Data uji berupa file YUV 4:2:0 dengan spesifikasi:

- Resolusi: 640x480 piksel (format QCIF)
- Jumlah frame: 150 *frame*
- Format: YUV420p
- Sumber: Dataset video konferensi umum atau sintetis
- Data uji digunakan sebagai masukan *pipeline* untuk menguji kinerja *framework* dalam memproses aliran video.

2.5 Perancangan Sistem

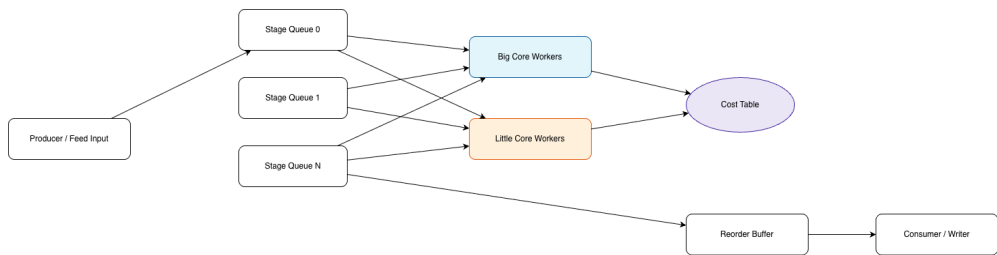
Perancangan sistem ini bertujuan untuk membangun sebuah *framework pipeline multithreading* yang bersifat adaptif terhadap heterogenitas *hardware*. *Framework* ini dirancang untuk menyeimbangkan beban kerja secara dinamis pada arsitektur asimetris (*big.LITTLE*) menggunakan metode *Runtime Cost Estimation (RCE)*.

2.5.1 Gambaran Umum Arsitektur

Secara umum, *framework* ini mengadopsi pola desain *Producer-Consumer* yang dimodifikasi dengan mekanisme *Pull-Based Scheduling*. Berbeda dengan *pipeline* tradisional yang menetapkan satu *thread* tetap untuk satu tahap (*stage*), *framework* ini menggunakan *Worker Pool* yang fleksibel. Setiap *worker (thread)* memiliki kemampuan untuk mengeksekusi tahap mana pun dalam *pipeline* secara dinamis berdasarkan estimasi beban kerja yang telah di kalkulasi.

Arsitektur sistem terdiri dari lima komponen utama:

- Producer Module*: Bertugas memasukkan data tugas (*frame video*) ke dalam antrean awal *pipeline*.
- Stage Queues*: Kumpulan antrean terbatas (*bounded buffer*) yang menampung tugas pada setiap tahap *pipeline*.
- Worker Pool*: Sekumpulan *thread* pekerja yang dibagi berdasarkan tipe inti prosesor (*big core* dan *little core*).
- RCE Engine*: Modul penghitung estimasi biaya komputasi yang menjadi dasar pengambilan keputusan.
- Consumer Module*: Bertugas mengumpulkan hasil akhir dan menjaga urutan output melalui *Reorder Buffer*.



Gambar 4. Gambaran Umum Arsitektur

2.5.2 Perancangan Komponen Utama

a. Worker Pool Asimetris

Komponen ini menggantikan model *thread-per-stage* yang kaku. *Worker pool* dirancang dengan mempertimbangkan topologi arsitektur asimetris dalam hal ini seperti pada Orange Pi 5 (RK3588S).

- 1) Klasifikasi *Thread*: *Thread* pekerja dibagi menjadi dua kelompok saat inisialisasi: *Big Workers* dan *Little Workers*.
- 2) *CPU Affinity*: Setiap kelompok *thread* diikatkan secara programatis ke inti fisik yang sesuai.
 - *Big Workers* diikat ke *core* 0-3 (Cortex-A76).
 - *Little Workers* diikat ke *core* 4-7 (Cortex-A55).

Hal ini memastikan bahwa *scheduler* memiliki kendali penuh atas penempatan tugas di tingkat hardware.

b. Engine Runtime Cost Estimation (RCE)

RCE adalah komponen inovatif yang menggantikan kebutuhan *offline profiling* tradisional. Berbeda dengan metode adaptif yang memonitor setiap iterasi, RCE menggunakan pendekatan kalibrasi awal (*initial calibration*) untuk meminimalkan *overhead runtime*.

- 1) Mekanisme Pengukuran: Pengukuran waktu eksekusi aktual (t_{actual}) hanya dilakukan sekali pada fase awal eksekusi (proses *warm-up*), di mana sistem memproses satu sampel data awal melalui seluruh tahap *pipeline*.
- 2) Penentuan Biaya: Hasil pengukuran dari proses awal ini langsung digunakan untuk mengisi tabel estimasi biaya (*cost*) pada *shared memory*.
- 3) Efisiensi Komputasi: Dengan menerapkan estimasi biaya yang ditetapkan di awal, sistem tidak perlu melakukan pengukuran waktu pada *frame-frame* berikutnya (*steady-state*). Hal ini secara signifikan mengurangi beban komputasi sistem, menghilangkan *overhead* pemanggilan fungsi *timing* di dalam *loop* utama pemrosesan.

c. Adaptive Task Scheduler

Scheduler adalah logika keputusan yang berjalan di setiap *worker thread*. Berbeda dengan *scheduler* standar (FIFO), *scheduler* ini menggunakan strategi *cost-aware*:

- 1) Logika *big core*: memprioritaskan pengambilan tugas dari *stage queue* dengan nilai *cost* tertinggi (tugas berat). Ini memastikan *core* yang kuat menangani bagian yang paling sulit.
 - 2) Logika *little core*: Memprioritaskan pengambilan tugas dari *stage queue* dengan nilai *cost* terendah (tugas ringan). Ini memastikan *core* hemat daya tidak kewalahan.
 - 3) *Fallback Mechanism*: Jika antrean prioritas kosong, *worker* dapat mengambil tugas dari antrean mana pun untuk mencegah *starvation* (memecahkan *thundering herd problem*).
- d. *Reorder Buffer* dan *Backpressure*
- 1) *Reorder Buffer*: Karena sifat eksekusi paralel yang *out-of-order* (tugas bisa selesai tidak berurutan), komponen ini menyimpan hasil sementara sebelum ditulis ke *disk*. *Consumer thread* akan menunggu hingga *frame* dengan ID berurutan tiba sebelum melakukan *write*.
 - 2) *Backpressure Control*: Mekanisme sinkronisasi kondisional yang memblokir *producer* jika antrean penuh. Ini mencegah *memory overflow* dan memaksa sistem menunggu hingga ada ruang kosong, menjaga stabilitas sistem secara alami.

2.5.3 Integrasi Studi Kasus (FSRCNN)

Untuk memvalidasi desain *framework*, aplikasi FSRCNN dipetakan ke dalam arsitektur *pipeline* sebagai berikut:

- a. Pemecahan Model: Model FSRCNN dibagi menjadi 8 tahap, di mana setiap tahap merepresentasikan satu lapisan jaringan saraf.
- b. *Callback Registration*: Fungsi pemrosesan setiap layer (mulai dari *Feature Extraction* hingga *Deconvolution*) didaftarkan sebagai fungsi *callback* ke dalam *array stages* pada konfigurasi *framework*.
- c. Eksekusi: Data piksel video mengalir melewati *stage queue* 0 hingga *stage queue* 7. *Worker* secara dinamis mengambil data ini, memprosesnya sesuai *callback* yang terdaftar, lalu mendorongnya ke antrean berikutnya. Lapisan *deconvolution* (layer 8) yang berat akan secara otomatis dikerjakan oleh *big core* berkat mekanisme RCE.

Dengan perancangan sistem ini, diharapkan *framework* mampu mendistribusikan beban kerja secara proporsional, memaksimalkan utilisasi *big core*, dan mengoptimalkan efisiensi energi *little core* tanpa intervensi manual pengguna.

2.6 Definisi Operasional Variabel

Tabel 4. Definisi Operasional Variabel

Variabel	Definisi Operasional	Metode Pengukuran
<i>Throughput</i>	Jumlah <i>frame</i> yang berhasil diproses per detik	$\frac{\text{total_frames}}{\text{total_processing_time}}$ diukur dengan fungsi <code>gettimeofday</code>
Utilisasi <i>CPU</i>	Rata-rata persentase waktu aktif seluruh <i>core</i> selama eksekusi	Menggunakan <code>perf stat -e cpu-clock</code> atau <code>mpstat</code> pada interval 1 detik
<i>Latency per frame</i>	Waktu yang dibutuhkan dari <i>feed</i> hingga output untuk setiap <i>frame</i>	Pengukuran dengan <code>gettimeofday</code> saat <i>feed</i> dan saat <i>consumer</i> menulis
<i>Reorder Overhead</i>	Jumlah <i>frame</i> yang harus ditahan di <i>reorder buffer</i> sebelum output	Log internal <i>framework</i> yang mencatat <i>frame</i> yang tidak langsung dapat ditulis
<i>Speedup</i>	Rasio <i>throughput framework</i> dinamis terhadap <i>pipeline</i> statis	$\frac{\text{throughput_dinamis}}{\text{throughput_statis}}$

2.7 Skenario Percobaan

Percobaan dirancang untuk menguji hipotesis bahwa *framework* berbasis *Runtime Cost Estimation (RCE)* dapat meningkatkan utilisasi sumber daya dan *throughput* dibandingkan dengan *pipeline* statis. Skenario yang diuji:

Tabel 5. Skenario Percobaan

Skenario	Konfigurasi	Tujuan
A (<i>Baseline</i>)	<i>Pipeline</i> statis dengan 8 thread (1 thread per layer)	Menetapkan <i>baseline</i> kinerja
B	<i>Framework</i> dengan 4 <i>worker</i>	Menguji skalabilitas pada jumlah <i>worker</i> di bawah jumlah <i>core</i>
C	<i>Framework</i> dengan 8 <i>worker</i>	Menguji skalabilitas pada jumlah <i>worker</i> setara <i>core</i> fisik

D	<i>Framework</i> dengan 12 <i>worker</i> (<i>oversubscription</i>)	Menguji batas <i>throughput</i> dan efek <i>oversubscription</i>
---	--	--

2.8 Analisis Data

Analisis data dilakukan dengan pendekatan kuantitatif deskriptif dan komparatif.

Langkah-langkah analisis:

- a. Perhitungan statistik deskriptif: rata-rata, standar deviasi, dan rentang nilai untuk setiap metrik pada setiap skenario.
- b. Perbandingan kinerja: menggunakan grafik batang untuk membandingkan *throughput* dan utilisasi CPU antarskenario.
- c. Analisis *speedup*: menghitung rasio peningkatan *throughput framework* dinamis terhadap *baseline*.
- d. Analisis *bottleneck*: mengidentifikasi tahap yang menjadi *bottleneck* melalui distribusi waktu eksekusi per tahap.

Hasil analisis akan menjawab rumusan masalah penelitian dan membuktikan atau menggugurkan hipotesis yang diajukan.

DAFTAR PUSTAKA

- Adnan, A., 2023. Performance evaluation on work-stealing featured parallel programs on asymmetric performance multicore processors. *Array* 19, 100311. doi:10.1016/j.array.2023.100311.
- Al-Shaikh, A., Shaheen, A., Rasmi Al-Mousa, M., Alqawasmi, K., Al Sherideh, A.S., dan Khattab, H., 2023. A Comparative Study on the Performance of 64-bit ARM Processors. *International Journal of Interactive Mobile Technologies (iJIM)* 17 (13): 94–113. <https://doi.org/10.3991/ijim.v17i13.39395>.
- Amdahl, G. M., 1967. Validity of the single-processor approach to achieving large-scale computing capabilities. *Proceedings of AFIPS Conference*, 483-485.
- Annisa, Adnan, and Zainuddin Z, 2025. Enhancing Computational Efficiency: Transitioning from Serial to Parallel Programming for Low-to-High Resolution Video Reconstruction Using FSRCNN on AMP Architecture. *IEEE Proceedings of 2025 IEEE International Conference on Artificial Intelligence and Mechatronics Systems (AIMS)*. doi: 10.1109/AIMS66189.2025.11229650.
- Arm., 2026. big.LITTLE: Balancing Power Efficiency and Performance, Arm Technologies. Available at <https://www.arm.com/technologies/big-little>. [Accessed on 1 April 2026]
- Chronaki, K., Rico, A., Casas, M., Moretó, M., Badia, R. M. and Ayguadé, E., 2017. Task scheduling techniques for asymmetric multi-core systems. *IEEE Transactions on Parallel and Distributed Systems* 28(7). 2074–2087. doi:10.1109/TPDS.2016.2633347.
- OECD., 2015. Frascati Manual 2015: Guidelines for Collecting and Reporting Data on Research and Experimental Development, The Measurement of Scientific, Technological and Innovation Activities. Retrieved from https://www.oecd.org/content/dam/oecd/en/publications/reports/2015/10/frascati-manual-2015_g1g57dcb/9789264239012-en.pdf.
- Cho, S., Lee, H., and Kim, J., 2025. LibraPIM: Dynamic Batch Offloading for Efficient Heterogeneous Computing with Processing-in-Memory. *IEEE Proceedings of 2025 34th International Conference on Parallel Architectures and Compilation Techniques (PACT 2025)*. doi: 10.1109/PACT65351.2025.00016.
- Dong, C., Loy, C. C., He, K., & Tang, X. 2015. Image Super-Resolution Using Deep Convolutional Networks (arXiv:1501.00092). arXiv. <https://doi.org/10.48550/arXiv.1501.00092>
- Dong, C., Loy, C. C., & Tang, X. 2016. Accelerating the Super-Resolution Convolutional Neural Network. In B. Leibe, J. Matas, N. Sebe, & M. Welling (Eds), *Computer Vision – ECCV 2016*. 9906.

- 391–407. Springer International Publishing.
https://doi.org/10.1007/978-3-319-46475-6_25.
- Gao, Y., Zhang, Z., Donta, P. K., Dehury, C. K., Wang, X., and Niyato, D., 2025. Optimizing Multi-DNN Inference on Mobile Devices through Heterogeneous Processor Co-Execution. *IEEE Transactions on Mobile Computing*, Early Access, 1–16. doi:10.1109/TMC.2025.3647031.
- Gustafson, J. L. 1988. Reevaluating Amdahl's Law. *Communications of the ACM*, 31(5), 532–533.
- Harlap, A., Narayanan, D., Phanishayee, A., Seshadri, V., Devanur, N., Ganger, G., and Gibbons, P., 2018. PipeDream: Fast and efficient pipeline parallel DNN training. *arXiv preprint arXiv:1806.03377*. doi:10.48550/arXiv.1806.03377.
- Hill, M. D., and Marty, M. R., 2008. Amdahl's Law in the Multicore Era. *Computer. IEEE Proceedings of 2008 14th International Symposium on High Performance Computer Architecture*. 33–38. doi: 10.1109/HPCA.2008.4658638
- Hu, Y., Imes, C., Zhao, X., Kundu, S., Beerel, P. A., Crago, S. P., and Walters, J. P., 2021. Pipeline parallelism for inference on heterogeneous edge computing. *arXiv preprint arXiv:2110.14895*. doi:10.48550/arXiv.2110.14895.
- Jiang, J., Huang, Z., and Huang, D., 2023. Hierarchical model parallelism for optimizing inference on many-core CPUs. *ACM Transactions on Architecture and Code Optimization* 20(3), Article 42, 1–21. doi:10.1145/3605149.
- Joseph, G., and Jayakumar, E. P., 2024. Quantized Neural Network Architecture for Hardware-Efficient Real-Time 4K Image Super-Resolution. *IEEE Proceedings of 2024 28th International Symposium on VLSI Design and Test (VDATE)*. doi: 10.1109/VDATE63601.2024.10705704
- Min, Y. I. and Eom, Y. I., 2013. DANBI: Dynamic Scheduling of Irregular Stream Programs for Many-Core Systems. *Proceedings of the 22nd International Conference on Parallel Architectures and Compilation Techniques*. doi: 10.1109/PACT.2013.6618816.
- Mastoras, A., and Gross, T., 2018. Unifying fixed code and fixed data mapping for pipelined loops. *IEEE Transactions on Parallel and Distributed Systems*. 29(9), 2136–2149. doi: 10.1109/TPDS.2018.2817207
- Mastoras, A., 2019. Efficient Execution of Linear Pipelines. PhD. Thesis. ETH Zurich, Zurich, Switzerland.
- Moreno, A., Sikora, A., César, E., Sorribes, J. and Margalef, T., 2017. HeDPM: load balancing of linear pipeline applications on heterogeneous systems. *Journal of Supercomputing*. 73. 3738–3760. doi:10.1007/s11227-017-1971-4.

- Patterson, D.A., Hennessy, J.L., dan Hennessy, J.L., 2012. Computer organization and design: the hardware/software interface. Edisi 4th ed. Morgan Kaufmann, Waltham, MA. Retrived from <https://nsec.sjtu.edu.cn/data/MK.Computer.Organization.and.Design.4th.Edition.Oct.2011.pdf>
- Saad, A., Abd el-Raouf, O., Hadhoud, M. and Kafafy, A., 2026. QLSA-MOEAD integration for precision task scheduling in heterogeneous computing environments. Scientific Reports 16. 7194. doi:10.1038/s41598-026-36916-1.
- Silberschatz, A., Galvin, P. B. and Gagne, G., 2018. Operating System Concepts (ed. 10). Retrieved from https://opac.atmaluhur.ac.id/uploaded_files/temporary/DigitalCollection/YTlhNjlyYzNlMjhiMjZmNDMyZmYyMjliYTZhYjhmNDkwOWQzNDM2Mg==.pdf.
- Tanenbaum, A. S., and Bos, H., 2015. Modern Operating Systems. 4th ed. Pearson.
- Wang, M., Ding, S., Cao, T., Liu, Y., and Xu, F., 2021. AsyMo: scalable and efficient deep-learning inference on asymmetric mobile CPUs. MobiCom '21: Proceedings of the 27th Annual International Conference on Mobile Computing and Networking, New Orleans, LA, USA. 215–228. doi:10.1145/3447993.3448625.
- Wang, S., Ananthanarayanan, G., Zeng, Y., Goel, N., Pathania, A., and Mitra, T., 2020. High-throughput CNN inference on embedded ARM Big.LITTLE multicore processors. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 39(10), 2254–2267. doi:10.1109/TCAD.2019.2944584.
- Yu, T., Petoumenos, P., Janjic, V., Leather, H., and Thomson, J., 2020. COLAB: Collaborative Multi-Factor Scheduler for Asymmetric Multicore Processors. IEEE International Symposium on High-Performance Computer Architecture (HPCA). 268–279. doi:10.1145/3368826.3377915.
- Yuan, Z., Wang, X., Nie, Y., Tao, Y., Li, Y., Shao, Z., Liao, X., Li, B., and Jin, H., 2025. DynPipe: Toward Dynamic End-to-End Pipeline Parallelism for Interference-Aware DNN Training. IEEE Transactions on Parallel and Distributed Systems, 36(11), 2366–2380. doi:10.1109/TPDS.2025.3605491.
- Yoo, S., 2015. An empirical validation of power-performance scaling: DVFS vs. multi-core scaling in big.LITTLE processor. IEICE Electronics Express 12(8), 1–9. doi:10.1587/elex.12.20150236.

LAMPIRAN

Lampiran 1. *State of the Arts*

No	Judul Karya Ilmiah	Objek dan Permasalahan	Metode Penyelesaian	Kinerja
1.	Judul: Framework Penjadwalan Dinamis Adaptif berbasis <i>Runtime Cost Estimation</i> untuk Pipeline Multithreading pada Arsitektur Asimetris (Studi Kasus: Fast Super-Resolution Convolutional Neural Network). Penulis: M. Hamdani Ilham Latjoro Tahun: 2026	Pipeline inferensi FSRCNN pada arsitektur asimetris (Orange Pi 5). Permasalahan: ketidakseimbangan beban antarlapisan menyebabkan utilisasi CPU tidak optimal pada pipeline statis.	Menjadwalkan sumber daya CPU secara adaptif berbasis <i>Runtime Cost Estimation</i> pada <i>pipeline multithreading</i> di arsitektur asimetri	

2.	Judul: COLAB: Collaborative Multi-Factor Scheduling for Asymmetric Multicore Processors. Penulis: Yu et al. Tahun: 2020	Penjadwalan thread pada prosesor asimetris (<i>big.LITTLE</i>) untuk meningkatkan performa dan efisiensi energi. Permasalahan: sulit memprediksi performa thread pada tipe inti yang berbeda dan mengidentifikasi <i>bottleneck</i>.	Mengusulkan scheduler kolaboratif multifaktor yang mengestimasi performa setiap thread pada setiap tipe inti, mengidentifikasi pola komunikasi, dan mengidentifikasi <i>thread</i> yang menjadi <i>bottleneck</i>.	Meningkatkan performa hingga 21% dan efisiensi energi hingga 18% dibandingkan dengan scheduler Linux standar pada workload campuran.
3.	Judul: HeDPM: Heterogeneous Dynamic Pipeline Mapping for Parallel Applications. Penulis: Moreno et al. Tahun: 2017	Pemetaan pipeline aplikasi paralel pada sistem heterogen. Permasalahan: ketidakseimbangan beban antartahap pipeline pada lingkungan heterogen.	Algoritma dinamis yang melakukan replikasi (menduplikasi tahap lambat di beberapa prosesor) dan gathering (menggabungkan tahap cepat pada prosesor yang sama) dengan mempertimbangkan kapasitas komputasi dan biaya komunikasi.	Meningkatkan kinerja pipeline Ferret pada <i>benchmark</i> PARSEC hingga 30% dibandingkan dengan pendekatan statis.

4.	Judul: DANBI: Dynamic Scheduling of Irregular Stream Programs for Many-Core Systems Penulis: Min et al. Tahun: 2013	Penjadwalan program stream irregular pada sistem many-core. Permasalahan: ketidakseimbangan beban akibat pola aliran data tidak beraturan.	Penjadwal dynamic load-balancing yang mengeksplorasi hubungan producer-consumer dalam program stream, menghindari thundering-herd problem, dan beradaptasi secara probabilistik.	Skalabilitas hampir linier pada server 40-core, mengungguli <i>runtime</i> paralel mutakhir hingga 2,8 kali lipat.
5.	Judul: Task Scheduling Techniques for Asymmetric Multi-core Systems. Penulis: Chronaki et al. Tahun: 2017	Penelitian ini berfokus pada arsitektur asymmetric multi-core yang bertujuan meningkatkan efisiensi energi melalui penggunaan berbagai jenis inti prosesor (inti cepat dan lambat). Permasalahannya adalah sulitnya memetakan tugas (task) secara dinamis ke inti yang sesuai secara efisien tanpa memerlukan <i>offline profiling</i>, terutama pada	Metode penyelesaian yang diusulkan mencakup dua teknik penjadwalan dinamis: CPATH dan HYBRID. CPATH mendeteksi jalur kritis aplikasi dengan melacak waktu eksekusi tugas secara runtime dan menetapkan prioritas berbasis biaya.	Pada sistem riil 8-inti, metode ini meningkatkan performa hingga 1,45 kali. Dalam simulasi sistem 32-inti, peningkatan performa mencapai 2,1kali.

		aplikasi dengan ketergantungan tugas yang kompleks.		
6.	Judul: PipeDream: Fast and Efficient Pipeline Parallel DNN Training Penulis: Harlap, A., et al. Tahun: 2018	Permasalahan utama dalam metode data-paralel tradisional adalah tingginya hambatan komunikasi saat menangani model besar atau bandwidth jaringan yang terbatas, yang dapat menghabiskan hingga 85% waktu pelatihan. Selain itu, model paralel tradisional sering kali menyebabkan rendahnya utilitas sumber daya komputasi.	Peneliti mengembangkan PipeDream, sebuah sistem yang menerapkan pipeline parallelism. Metode ini membagi lapisan DNN menjadi beberapa tahap (stages) yang didistribusikan ke berbagai pekerja, di mana setiap pekerja memproses minibatch yang berbeda secara bersamaan dalam sebuah pipa (pipeline)	PipeDream terbukti hingga 5 kali lebih cepat dalam mencapai target akurasi dibandingkan dengan metode data-paralel. Sistem ini mampu mengurangi volume komunikasi hingga 95% untuk model DNN besar dan memungkinkan tumpang tindih sempurna antara tugas komunikasi dan komputasi.

7.	Judul: High-Throughput CNN Inference on Embedded ARM <i>big.LITTLE</i> Multicore Processors (Pipe-it) Penulis: Wang, S., et al. Tahun: 2020	Penelitian ini berfokus pada optimasi inferensi CNN pada prosesor heterogen ARM <i>big.LITTLE</i>. Masalah utamanya adalah <i>overhead</i> komunikasi interklaster yang tinggi saat menggunakan paralelisasi tingkat kernel (HMP), yang justru menurunkan <i>throughput</i> karena latensi akses memori pada bus CCI.	Untuk mengatasinya, dikembangkan kerangka kerja Pipe-it yang menerapkan strategi <i>Layer-level splitting</i> dalam desain pipa (pipeline). Alih-alih membagi satu kernel ke semua <i>core</i>, Pipe-it membagi lapisan-lapisan CNN ke berbagai klaster <i>core</i> agar dapat memproses beberapa gambar secara paralel dalam satu aliran data.	Pipe-it meningkatkan <i>throughput</i> rata-rata sebesar 39% dibandingkan dengan hanya menggunakan klaster <i>core</i> Big.
----	---	--	--	--

8.	Judul: AsyMo: Scalable and Efficient Deep-Learning Inference on Asymmetric Mobile CPUs Penulis: Wang, M. et al. Tahun: 2021	Penelitian ini berfokus pada inferensi Deep Learning (DL) pada CPU seluler dengan arsitektur multiprosesor asimetris (AMP). Masalah utamanya adalah skalabilitas performa yang buruk dan ketidakefisienan energi. Hal ini disebabkan oleh pembagian tugas yang tidak tepat, distribusi beban kerja yang tidak seimbang antara <i>core</i> besar dan kecil, serta ketidaktahuan sistem terhadap perilaku model dalam mengatur frekuensi CPU.	Peneliti mengusulkan AsyMo, sebuah teknik yang memanfaatkan determinisme eksekusi DL untuk membangun rencana eksekusi luring yang optimal.	AsyMo memberikan peningkatan signifikan pada berbagai kerangka kerja seperti TensorFlow dan TFLite. Hasilnya menunjukkan peningkatan performa hingga 46% dan efisiensi energi 37% untuk model dominan konvolusi.
----	---	--	---	---

9.	Judul: EdgePipe: Optimal Pipeline Parallelism for Heterogeneous Edge Inference Penulis: Hu, Y. et al. Tahun: 2021	Penelitian ini berfokus pada akselerasi inferensi Deep Neural Network (DNN) skala besar, seperti Vision Transformer (ViT), pada perangkat edge yang memiliki keterbatasan sumber daya. Permasalahan utamanya adalah model-model ini terlalu berat untuk kapasitas memori dan komputasi perangkat tunggal, sementara solusi paralelisme yang ada umumnya dirancang untuk lingkungan pusat data yang homogen dan tidak mempertimbangkan heterogenitas jaringan serta perangkat di lingkungan edge.	Penulis mengusulkan EdgePipe, sebuah kerangka kerja terdistribusi yang memanfaatkan pipeline parallelism untuk membagi model menjadi beberapa tahap eksekusi.	Kinerja EdgePipe berhasil mencapai peningkatan kecepatan (<i>speedup</i>) hingga 10,59 kali untuk ViT-Large dan 11,88 kali untuk ViT-Huge menggunakan 16 perangkat edge tanpa kehilangan akurasi.
----	---	---	--	--

10.	Judul: Hierarchical Model Parallelism for Optimizing Inference on Many-core Processor via Decoupled 3D-CNN Structure. Penulis: Jiang, J., et al. Tahun: 2023	Optimasi inferensi CNN pada prosesor <i>many-core</i> (ARM). Permasalahan: Keterbatasan memori dan cache pada prosesor <i>many-core</i> saat menjalankan model CNN yang kompleks secara berurutan.	Mengusulkan pendekatan Hierarchical Model <i>Parallelism</i> yang memecah struktur CNN (<i>Decoupled</i> CNN) menjadi sub-graf yang lebih kecil untuk dieksekusi paralel, memaksimalkan utilisasi cache.	Meningkatkan kecepatan inferensi 3D-CNN secara signifikan dengan memanfaatkan paralelisme tingkat model pada arsitektur <i>many-core</i>.
-----	--	---	---	--

11.	Judul: Performance evaluation on work-stealing featured parallel programs on asymmetric performance multicore processors. Penulis: Adnan Tahun: 2023	Penelitian ini berfokus pada Asymmetric Performance Multicore Processors (AMPs) seperti Intel Core i5 1240P yang menggabungkan P-Cores (performa tinggi) dan E-Cores (efisiensi energi). Masalah utamanya adalah hukum Amdahl tradisional tidak lagi akurat karena mengasumsikan kecepatan semua inti seragam. Selain itu, terdapat masalah ketidakseimbangan beban kerja (load imbalance) di mana inti yang lebih cepat sering mengganggu menunggu inti yang lambat jika distribusi kerja tidak dilakukan secara proporsional.	Metode yang diusulkan adalah memperbarui persamaan hukum Amdahl dengan mengintegrasikan <i>speedup factor (sf)</i> untuk mengestimasi performa secara lebih tepat. Untuk mengatasi ketidakseimbangan beban, digunakan framework OpenCilk yang menerapkan mekanisme work-stealing pada tingkat pengguna (user-level).	Hasil evaluasi menunjukkan bahwa <i>P-Core</i> secara empiris dua kali lebih cepat daripada <i>E-Core</i> ($sf \approx 2$) pada frekuensi yang sama. Penggunaan work-stealing terbukti efektif karena <i>P-Core</i> mampu mempertahankan nilai IPC-nya tanpa mengalami degradasi performa saat bekerja dalam mode asimetris.
-----	--	--	---	--

12.	Judul: Optimizing Multi-DNN Inference on Mobile Devices through Heterogeneous Processor Co-Execution. Penulis: Gao, Y., et al. Tahun: 2025	Penelitian ini berfokus pada optimasi inferensi multi-DNN pada perangkat seluler yang menggunakan prosesor heterogen (CPU, GPU, DSP, NPU) . Masalah utama yang diidentifikasi adalah rendahnya pemanfaatan perangkat keras pada kerangka kerja saat ini, fragmentasi subgraf berlebihan yang meningkatkan beban memori dan penjadwalan, serta penurunan performa drastis akibat panas (thermal throttling).	Metode penyelesaian yang diusulkan adalah ADMS (Advanced Multi-DNN Model Scheduling) . Strategi ini menggunakan pemisahan subgraf adaptif dengan kontrol granularitas perangkat keras untuk mengurangi fragmentasi.	ADMS berhasil mengurangi latensi inferensi hingga 4,04 kali dibandingkan dengan TFLite dan meningkatkan efisiensi energi sebesar 24,2% dibandingkan dengan sistem Band.
-----	--	--	--	--

13.	Judul: DynPipe: Toward Dynamic End-to-End Pipeline Parallelism for Interference-Aware DNN Training Penulis: Yuan et al. Tahun: 2025	Objek penelitian ini adalah pelatihan model Deep Neural Network (DNN) skala besar seperti BERT, GPT, dan VGG melalui paralelisme pipa terdistribusi. Permasalahan utamanya adalah sistem yang ada bersifat statis dan tidak efisien dalam menghadapi lingkungan komputasi dinamis, seperti interferensi tugas eksternal atau gangguan jaringan. Selain itu, sistem tradisional sering mengabaikan dampak keusangan model (staleness) dan <i>overhead</i> komunikasi terhadap kecepatan konvergensi akhir.	Peneliti mengusulkan DynPipe, sebuah kerangka kerja paralelisme pipa asinkron yang adaptif. DynPipe menerapkan perencanaan end-to-end yang menyeimbangkan kecepatan perangkat keras dengan efisiensi statistik konvergensi.	DynPipe mempercepat waktu-ke-akurasi sebesar 1,5–3,4 kali dibandingkan sistem lainnya
-----	---	--	--	--

14.	Judul: LibraPIM: Dynamic Batch Offloading for Efficient Heterogeneous Computing with Processing-in-Memory Penulis: Cho S., et al. Tahun: 2025	Komputasi heterogen dengan <i>Processing-in-Memory</i> (PIM). Permasalahan: kontensi sumber daya bank DRAM dan ketidakseimbangan beban antara xPU dan PIM.	Dynamic Batch Offloading (DBO) yang memprediksi beban kerja dan memindahkan sebagian beban secara <i>fine-grained</i> dari <i>bottleneck</i> ke mitra <i>idle</i>, serta <i>Dual-Path Execution (DEX)</i> memisahkan jalur data.	Percepatan rata-rata 6,2 kali, peningkatan utilisasi hingga 38% untuk PIM dan 77% untuk NPU.
-----	---	---	---	---

15.	Judul: Enhancing Computational Efficiency: Transitioning from Serial to Parallel Programming for Low-to-High Resolution Video Reconstruction Using FSRCNN on AMP Architecture. Penulis: Annisa et al. Tahun: 2025	Implementasi paralel FSRCNN pada prosesor asimetris (<i>big.LITTLE</i>). Permasalahan: <i>Bottleneck</i> komputasi yang signifikan pada <i>layer 8</i> (tahap dekonvolusi) dan kinerja suboptimal pada skenario alokasi inti statis (hibrida).	Analisis perbandingan skenario alokasi statis (Big saja, Little saja, dan Hybrid) untuk mengidentifikasi titik <i>bottleneck</i>. Penelitian ini mengonfirmasi bahwa ketergantungan pada alokasi tetap menyebabkan ketidakseimbangan beban.	Skenario 4 inti Big menghasilkan kualitas video dan waktu komputasi terbaik. Skenario Hybrid menawarkan keseimbangan, namun tidak mampu mengatasi <i>bottleneck</i> pada <i>layer 8</i> secara efektif.
-----	---	---	--	--

16.	Judul: QLSA-MOEAD: Adaptive Multi-Objective Task Scheduling in Heterogeneous Computing Penulis: Saad, A., et al. Tahun: 2026	Penelitian ini berfokus pada penjadwalan tugas (<i>workflow</i>) dalam lingkungan komputasi heterogen yang melibatkan CPU, GPU, dan FPGA. Tantangan utamanya adalah ketergantungan tugas yang kompleks (DAG) serta tujuan optimasi yang saling bertentangan, yaitu meminimalkan makespan (waktu penyelesaian), memaksimalkan pemanfaatan sumber daya, dan menekan konsumsi energi.	Peneliti mengusulkan kerangka kerja hybrid bernama QLSA-MOEAD yang mengintegrasikan tiga mekanisme.	QLSA-MOEAD mencapai peningkatan performa hingga 4,3 kali lipat dibandingkan dengan metode lainnya. Selain itu, sistem ini sangat adaptif terhadap tugas dinamis dengan waktu respons sub-milidetik (0,80–1,70 ms)
-----	--	---	--	--